

Bridging the Namespace Canyon

The problem we keep paying for

Over the last decade, the software ecosystem has collectively paid a **huge and recurring cost** for choosing

configuration and policy formats that deliberately omit namespaces. JSON and YAML achieved popularity

by being simpler than XML, but that simplicity came with a hidden tax:

- * Meaning is carried by **strings**, not identifiers
- * Semantics are implicit, informal, and repeatedly re■negotiated
- * Every project re■discovers the same ambiguities
- * Governance, interoperability, and long■term maintenance suffer

Industries built on formal standards — finance, insurance, healthcare, government — feel this pain acutely. Standards

live in namespaces. When namespaces are stripped away, meaning erodes.

This is the **namespace canyon**: on one side, rigorously defined standards (XSD, RDF, OWL); on the other, lightweight

configuration formats that cannot natively express identity or scope.

Why “it’s just configuration” is the wrong answer

The common defense is:

> “Policy files are just configurations; they don’t need namespaces.”

That argument fails when:

- * policy decisions are driven by **which standard namespace** is in play
- * multiple standards coexist in a single transformation
- * mappings must be **auditable, reproducible, and explainable**
- * the policy itself becomes part of provenance

At that point, a policy document is no longer “just config data.” It is a **formal artifact** whose meaning must survive

time, teams, and tools.

The W3C answer hiding in plain sight: YAML■LD

The Web community has already faced this problem.

The **W3C JSON■LD Community Group** produced a formal specification for **YAML■LD**: a way to write

Linked Data *in* YAML syntax while preserving full RDF/IRI semantics.

Key idea:

- * YAML is used only as a **surface syntax**
- * Semantics come from **JSON■LD** via `@context``
- * Namespaces, prefixes, and IRIs are first■class

This is not a hack or a local convention. It is a documented, interoperable mapping with well-defined behavior.

What YAML■LD provides that plain YAML never can

With YAML■LD:

- * Namespaces are explicit and machine-interpretable
- * Identifiers are IRIs, not magic strings
- * Policy meaning can be validated, expanded, and reasoned over
- * The same document can be treated as YAML, JSON■LD, or RDF

In other words, YAML■LD **builds a bridge** across the namespace canyon instead of pretending it doesn't exist.

Why this matters for standards bodies like MISMO

When a standards organization publishes or consumes JSON without a namespace story, it unintentionally:

- * fragments meaning across implementations
- * pushes semantic responsibility onto downstream projects
- * it makes long-term governance harder, not easier

When a policy or configuration document is coded in YAML■LD, it allows:

- * domain standards to remain anchored to their namespaces
- * tooling to evolve without re-litigating meaning
- * demonstrations and pilots that don't accrue semantic debt
- * JSON compatibility is maintained because any valid JSON file is a valid YAML-LD document.

For MISO-to-RDF work, this is not theoretical. Mapping policy *must* be namespace-aware to be correct.

A practical, crawl-phase takeaway

Given that "Crawl, Walk, Run" is a common metaphor in software engineering, we will begin by crawling out of the namespace canyon JSON publications create.

We do not need to convert everything to RDF as part of publications.

We *do* need to stop pretending that namespace-free formats are neutral.

YAML-LD offers a pragmatic middle path:

- * familiar authoring experience
- * formal semantics
- * alignment with existing W3C standards

That bridge already exists. We just need to use it.

Standards scale when their identifiers travel intact across time, space, and implementations.

The industry does not need another format shift.

It needs its identifiers to travel intact.

This document captures a design principle, not a tooling mandate: when meaning matters, identifiers matter — and namespaces are how we keep meaning intact.

1] <https://www.w3.org/community/reports/json-ld/CG-FINAL-yaml-ld-20231206>